Assignment 3: CS-564 [Machine Learning]
Neural Networks

Deadline: November 6th, 2025

Instructions

- Code must be properly indented, commented, and well-structured.
- Marks will be deducted in case of plagiarism.
- All required files must be compressed and named as: ROLLNO1_ROLLNO2_ROLLNO3.zip. E.g. 22221CS15_2221CS16_2221CS16.zip
- Upload the assignment to https://forms.gle/GBfbcyPKTSyuD7b3A

Problem Statement

You are required to build a handwritten digit classification by implementing Artificial Neural Network. You need to train the model to correctly identify handwritten digits (0 to 9), from the provided datasets. The training will involve forward propagation, backward propagation and gradient descent.

Dataset

- Import the MNIST Handwritten Digits dataset. You can use the libraries like sklearn.datasets or keras.datasets.
- Total images: **Total images:** 70,000 (60,000 training, 10,000 testing)
- **Image size:** 28x28 pixels (grayscale)
- **Features:** 784 (from flattening the 28x28 image)
- **Classes:** 10 (one for each digit from 0 to 9)

Implementation Requirements

Data Preprocessing

- Normalize the pixel values using Min-Max scalar
- Convert the target labels into one-hot encoded vectors. For example [0,0,0,1,0,0,0,0,0,0] for 3. This will be necessary for training with SoftMax output layer.

Model Implementation

- Implement a neural network with at least one hidden layer. There will be 784 nodes in the input layer, one for each pixel. In the first hidden layer you can choose the number of nodes, say 128 and same applies for the other hidden layer. The output layer will contain 10 nodes, one for each output

- You can implement a function to initialize weights and biases. It can be small random values.
- You can implement the Sigmoid or ReLU activation function for the hidden layer and SoftMax activation function for the output layer.
- Define the forward propagation function that takes inputs and computes the network's output (class probabilities).
- Define the function that calculates cross-entropy loss.
- Derive and implement the backpropagation algorithm to compute the gradients of the loss with respect to all weights and biases in the network.
- You can use the Stochastic Gradient Descent (SGD) or Mini-Batch Gradient Descent to adjust the network's parameters.

Training and Testing

- Create a training loop that should iterate over the training data for a fixed number of epochs.
- You can use mini batches (e.g., 32 or 64 samples per batch) to update your gradients.
- Evaluate the model performance (loss and accuracy) on a validation set. You can split 10% of training data for this.
- After the training, evaluate your final model on test set.

Evaluation Metrics

- Calculate the following:
- Confusion Matrix
- Accuracy
- Precision
- F1 Score

Restrictions

- Don’t use pre-built ANN libraries like sklearn.neural_network, keras, TensorFlow or Pytorch, instead use numpy, matplotlib. You can use sklearn.datasets, sklearn.preprocessing, sklearn.metrics, sklearn.model_selection.